1 Installation Instructions

This section will focus on installing Python 3.10. There are many ways to setup python. Here, we will use *Miniconda* since that is a painless way to do it.

1.1 Setup using pool computers

There is a computer pool with computer that can be used to run the code of the exercises. You can use the computers physically, or remotely via SSH.

1.1.1 Storage

If you want to train on the computers in the PC pool, you will need to put your installations and data into a shared folder which we create for you. Otherwise, the space in your home folder will not be enough. You will also need the access to be able to download the datasets.

To request access, please check our website.

1.1.2 Connecting via SSH

You can connect to the login server of the technical faculty via:
`ssh username@login.informatik.uni-freiburg.de` .

The first thing you should do once you are logged in, is to check if you can cd into `/project/cv-ws2425/username`. Note: The folder is invisible so you cannot use ls or tab key to see it. If not, make sure you contacted us to create your folder as described above. For other information or problems with your account you can find more information at `http://poolmgr.informatik.uni-freiburg.de`

It is recommended to set up key-based authentication and put the connection to your SSH config file. This is described in the following for Linux/Mac (but should work similarly for Windows). With this, you don't need to enter your password every time you log in:

```
ssh-keygen  # enter for example /home/username/.ssh/tf_rsa as name
ssh-copy-id -i /home/username/.ssh/tf_rsa lmb@login.informatik.uni-freiburg.de
# connect using the key:
ssh -i /home/username/.ssh/tf_rsa lmb@login.informatik.uni-freiburg.de
```

Next, it is recommended to put the connection to your SSH config file:

```
# Append the following to your ssh config (located at .ssh/config):
Host tflogin
HostName login.informatik.uni-freiburg.de
User username # put your username
Identityfile /home/username/.ssh/tf_rsa
Port 22
```

Now you should be able to connect to the login server via `ssh tflogin`.

Now you are logged in to the **login server. Note that you should not run anything on this server.** Instead, from the login server you should connect to one of the pool machines. The pool machines are named `tfpool25` to `tfpool46` and `tfpool51` to `tfpool63`. To connect to them, from the login server do for example: `ssh tfpool53` (exchange `tfpoolXX` for whichever machine you want to use). Note that tfpool machines are sometimes switched off. You have to try different machines until you find one that works (or you go to the pool physically and switch the machine on). You can also add pool machines to your SSH config:

```
# Append the following to your ssh config (located at .ssh/config):
Host tfpool53
HostName tfpool53
ProxyJump tflogin
User username # put your username
Identityfile /home/username/.ssh/tf_rsa
Port 22
# note: you have to add this for each tfpool machine that you want to use
```

Now you can login to the tfpool machine directly from your local machine by executing `ssh tfpool53`.

Before running something on the tfpool machine, make sure no one else is using the selected computer and the GPU is free. For this, you can use the following commands:

- See GPU load: `nvidia-smi`
- See logged in accounts: `who`
- View running processes: `top`

See the github repository for a script to find out which tfpool machines are switched on and free.

To keep your code running in the background, you can use screen:

- Start the session
  ```
  screen -S t01
  # run your experiment
  ```
- Detach from screen using: ctrl+a+d
- Login back into screen:
  ```
  screen -ls
  screen -r t01
  ```
- Write down on which computer you started your screen session.
- ctrl+esc to enter copy mode if you need to scroll up, navigate with arrow/page keys, copy output etc

When installing Miniconda make sure to install it into `/project/cv-ws2425/username/miniconda/`

Also move the directory containing the exercise code into `/project/cv-ws2425/username/`

1.2 Linux

1.2.1 Install Miniconda

- Download the Miniconda Linux installer: https://docs.conda.io/en/latest/miniconda.html Usually the 64-bit version is correct. Use `wget [link]` to download a file in the terminal.
- Open the terminal and `cd` into the folder containing *Miniconda3-latest-Linux-x86_ 64.sh*.
- Run `bash Miniconda3-latest-Linux-x86_64.sh`. Install and remember to select the correct installation directory when working with computers in the PC pool as described above. When asked by the installer to initialize Miniconda3, answer yes. This way, your `~/.bashrc` file will be modified so you have conda available at startup.
- Close and reopen your terminal window to make sure the changes take effect.
- Test your installation by running `conda --version` to see the its version.

1.3 Windows 10

1.3.1 Install Miniconda

- Download miniconda: https://docs.conda.io/en/latest/miniconda.html and install it. Usually the 64-bit version is correct.
- Open the *Anaconda Prompt (miniconda3)* you just installed.
- Run the command `python -V`. It should output `Python (version number)` or similar.

1.4 macOS

1.4.1 Install Miniconda

There are two ways to install Miniconda in macOS.

- Download `Miniconda3 MacOSX 64-bit pkg`: https://docs.conda.io/en/latest/miniconda.html
- Open *Miniconda3-latest-MacOSX-x86_ 64.pkg* in the Finder and follow the instructions on screen.
- When installation is complete, test it by opening a terminal and running `conda --version` to see its version.

OR

- Download `Miniconda3 MacOSX 64-bit bash`: https://docs.conda.io/en/latest/miniconda.html
- Open the terminal and `cd` into the folder containing *Miniconda3-latest-MacOSX-x86_ 64.bash*.
- Run `bash Miniconda3-latest-MacOSX-x86_64.bash`. Accept the default settings.
- Close and reopen your terminal window to make sure the changes take effect.
- Test your installation by running `conda --version` to see the its version.

2 Set up your environment

Next we will set up conda with a new environment named `cvenv` which uses Python 3.10. Run the following commands in your Miniconda prompt (Windows) or terminal (macOS, Linux):

```
conda --version
conda update conda -y
conda create --name cvenv python=3.10 -y
conda activate cvenv
conda env list
```

The last command should show you all installed environments (including `base` and the activated `cvenv` environment). This way, you have a separate environment to install everything for this course and avoid possible interference with/from other python projects.

Note: Whenever you start the miniconda prompt/terminal for this course, run `conda activate cvenv` to switch to the correct environment.

3 Run your first code

We have prepared hello world code for you to test your installation.

- Clone the exercise repository using git (see website).
- Open the Miniconda prompt (Windows) or terminal (macOS, Linux) and `cd` into the repository.
- Run `pip install -r requirements.txt` to install the required package.
- ‘cd‘ into folder ‘ex01‘

- Run `python hello.py` and check the output.

4 Install PyTorch

Later exercises require PyTorch.

In your miniconda environment, install PyTorch and TorchVision from here, the exact command to setup with GPU support depends on your system. On the pool computers use this command:
`conda install -y pytorch torchvision pytorch-cuda=11.8 -c pytorch -c nvidia`
Afterwards check if your GPU is detected (the command should print True):
`python -c 'import torch; print(torch.cuda.is_available())'`
In case you get the error `chardet not found`, run `pip install chardet`
See learn the basics and other tutorials to get started with PyTorch.

5 Optional: Get an IDE

It is possible to edit all your code in a simple text editor, however that is a lot harder than using a good tool. We recommend either *VSCode* or *PyCharm*.

5.1 PyCharm setup tutorial

Install *PyCharm Community Edition* from here: `https://www.jetbrains.com/pycharm/`
Start it and create a new python project.

Now you want to be able to debug in PyCharm, but also still run code in your command prompt. So you have to tell PyCharm to use your existing miniconda environment:

- First we have to choose the Python Interpreter:
  - *Windows, Linux*: Go to File → Settings → Project → Python Interpreter.
  - *macOS*: Go to PyCharm → Preferences → Project → Python Interpreter.
- With your project selected, click the wheel on the right of *interpreter* and select *Add...*
- Select *Conda Environment* on the left and *Existing environment* on the right.
- Click the 3 dots on the right of *interpreter* and input the path to the `python` interpreter of your `myenv` miniconda environment. To get the path:
  - *Linux, macOS*: Activate your conda environment in a terminal with `conda activate myenv` and then run `which python` to find the interpreter used by your environment.
  - *Windows*: Default path is `C:\Users\(yourname)\miniconda3\envs\myenv\python.exe`, unless you had changed it during setup.
- Also select *Make available to all projects* so you can use that environment everywhere.
- Confirm changes.

To find out how PyCharm works, try importing your code repository as a new project. Make sure the project interpreter is set correctly to your `myenv` environment.

Now under Run → Edit Configurations, setup the `hello.py` file correctly and run it within PyCharm. You can also try setting a breakpoint in the main function by clicking next to a line number and debug the script.

Note: It can be very helpful in the long run to tune the settings to your preferences and learn the important hotkeys to become a fast coder.

5.2 VSCode setup tutorial

- Install VSCode on your local machine. Open VSCode and install the SSH extension, the Python extension, and the Jupyter extension.
- In VSCode, connect via SSH to one of the tfpool machines (should work out of the box if you entered the things to your `.ssh/config` as described above; more details can be found at https://code.visualstudio.com/docs/remote/ssh).
- Once you are connected:
  - you can use the Terminal within VSCode to do things on the machine. For example, you could do all the miniconda and repository setup that is described above from here (in case you have not done it yet).
  - you can open the `cv-exercises` folder and edit, debug, run code
  - once you opened the folder, you need to select a python interpreter (press F1 and search for python interpreter); select the one from your `cvenv`
  - you can run code either using the terminal, or using the run functionality from VSCode (press the play button)